A Full Technical and Commercial Assessment of Serverless GPU Platforms for High-Throughput OCR Inference with ROlmOCR and vLLM

1.0 Executive Summary

This report provides a comprehensive technical and commercial analysis of leading serverless GPU platforms for deploying the ROlmOCR model via the vLLM inference engine. The objective is to identify the optimal solution balancing performance, total cost of ownership (TCO), and developer velocity for a high-throughput Optical Character Recognition (OCR) workload. The analysis is intended to guide strategic infrastructure decisions for deploying state-of-the-art, open-source AI models in a scalable and economically viable manner.

The primary technical hurdle in serverless GPU inference is managing "cold starts"—the latency incurred while provisioning a GPU, loading large model weights (like the approximately 8.29B parameters of ROlmOCR) into Video RAM (VRAM), and initializing the inference engine.[1] This challenge directly impacts both user experience in real-time applications and throughput efficiency in batch processing. A successful deployment hinges on selecting a platform that effectively mitigates this latency while maintaining a predictable and competitive cost structure.

The key findings of this assessment indicate a clear segmentation in the serverless GPU market. The landscape comprises three distinct categories: AI-native platforms (e.g., RunPod, Modal) offering specialized performance features and developer-centric workflows; general-purpose serverless platforms from hyperscalers (e.g., Google Cloud Run, Azure Container Apps) providing deep ecosystem integration and enterprise-grade SLAs; and managed API providers (e.g., Replicate) that prioritize ease of use and access to a broad model library. The analysis reveals that for this specific workload—a custom containerized model leveraging a high-performance inference engine—the AI-native platforms currently offer a superior blend of performance and cost-effectiveness.

The primary recommendation of this report is the selection of **RunPod** for a production deployment of ROlmOCR with vLLM. This platform offers an unparalleled combination of industry-leading low cold-start times (under 200ms with its FlashBoot technology), highly competitive per-second GPU pricing, zero data transfer fees, and robust, straightforward support for custom Docker containers. These features directly address the core technical and commercial requirements of the specified high-throughput OCR workload, making it the most balanced and strategically sound choice.

Secondary recommendations are provided for organizations with different strategic priorities:

- For teams prioritizing developer experience and rapid iteration within a Python-native ecosystem, **Modal** presents a compelling alternative. Its code-first approach abstracts away infrastructure complexity, and its recent implementation of GPU snapshotting makes it highly competitive on performance, despite a slightly different cost structure.
- For organizations deeply integrated into the Google Cloud ecosystem and requiring a fully managed, SLA-backed environment, **Google Cloud Run** is a viable option. However, decision-makers must be prepared for potentially higher and more variable cold-start latencies and a more complex, "à la carte" pricing model that includes separate charges for CPU, memory, and data egress.

The subsequent sections of this report detail the in-depth analysis supporting these conclusions. The assessment begins with a technical deconstruction of the ROlmOCR model and the vLLM engine to establish workload requirements. This is followed by a comparative technical and commercial assessment of the leading platforms, and the report concludes with a detailed TCO analysis and strategic implementation guidance for the recommended solution.

2.0 Analysis of the Target Workload: OlmOCR and ROlmOCR

A precise understanding of the target model is foundational to any infrastructure assessment. The model's architecture, size, and dependencies dictate the non-negotiable hardware and software requirements that platforms must meet. This section deconstructs the OlmOCR and ROlmOCR models to establish a clear technical baseline for the subsequent platform evaluation.

2.1 Model Architecture and Foundation

Both OlmOCR and its successor, ROlmOCR, are sophisticated Vision Language Models (VLMs) engineered specifically for the task of converting complex, image-based documents like PDFs and scanned images into clean, structured Markdown text.[5] Their purpose is to preserve the natural reading order and structural elements such as tables, equations, and lists, making them ideal for preparing datasets for Large Language Model (LLM) training or for powering document analysis applications.[5]

The models share a common lineage, being built upon the powerful Qwen-VL series of open-source models developed by Alibaba's Qwen team.[6] This foundation provides the core multi-modal capabilities required to interpret both the visual layout and the textual content of a document page.

- **OlmOCR**, developed by the Allen Institute for AI (AI2), has several versions fine-tuned from different Qwen-VL bases. The olmOCR-7B-0225-preview model is fine-tuned from Qwen2-VL-7B-Instruct [2], while the later olmOCR-7B-0725 version is fine-tuned from the improved Qwen2.5-VL-7B-Instruct base model.[3]
- **ROlmOCR**, developed by Reducto AI, explicitly builds upon the more recent Qwen2.5-VL-7B foundation, positioning itself as a performance-optimized successor to OlmOCR.[10]

The critical parameter for infrastructure planning is the model size. Both model families are based on a 7-billion-parameter architecture, with model cards specifying a size of **8.29B parameters**.[2] This substantial size is the primary driver of the significant VRAM and computational resources required for inference.

2.2 Resource Profiling and Hardware Requirements

The model's size and complexity translate directly into specific hardware prerequisites. Any platform under consideration must, at a minimum, meet these baseline requirements to be considered viable.

- **VRAM (Video RAM):** The most critical constraint is GPU memory. The model's weights, along with the Key-Value (KV) cache generated during inference, must fit into the GPU's VRAM. The official documentation and usage guides provide clear specifications:
  - The GitHub repository for a multi-modal OCR application using ROlmOCR states that the model requires **at least 16GB of VRAM**.[10]
  - A tutorial for deploying ROlmOCR on the Vast.ai platform reinforces this, but further recommends selecting an instance with **60GB of VRAM** to accommodate larger

documents and enable a wider context window for more complex analysis.[10]
    - A separate tutorial for installing the original OlmOCR model suggests using either an **NVIDIA RTX 4090 (24GB VRAM)** or an **RTX A6000 (48GB VRAM)**.[14]
    - This data establishes a definitive hardware floor: any candidate serverless platform must offer GPU instances with at least 16GB of VRAM, with a strong preference for options in the 24GB to 48GB+ range to ensure robust performance across diverse document types.
- **Compute Capability:** The underlying software, particularly the vLLM inference engine, relies on specific features of modern GPUs. vLLM's documentation specifies official support for NVIDIA GPUs with a **compute capability of 7.0 or higher**.[15] This includes architectures such as Volta (V100), Turing (T4, RTX 20xx), Ampere (A100, L4), and Hopper (H100).[15] While vLLM has emerging support for AMD ROCm and Intel XPU platforms, these are less mature and lack the broad ecosystem support of NVIDIA CUDA.[16] For a production deployment, relying on the well-supported NVIDIA ecosystem is the most prudent path, making the availability of these specific GPU types a key evaluation criterion.
- **Software Dependencies and Containerization:** The OlmOCR toolkit is noted for having complex Python dependencies that can be difficult to install in an existing environment.[5] The project authors strongly recommend using a clean environment and provide an official Docker image (alleninstituteforai/olmocr:latest) to encapsulate these dependencies and ensure a consistent, reproducible runtime.[5] This makes first-class support for deploying custom Docker containers a non-negotiable requirement for any selected platform.

2.3 Strategic Selection: Why ROlmOCR is Superior for Serverless Deployment

While both OlmOCR and ROlmOCR are capable models, a deeper analysis of their architectural differences reveals that ROlmOCR is the technically and commercially superior choice for a serverless deployment. This conclusion stems from a key optimization made by ROlmOCR's developers that directly addresses the primary cost and performance drivers of serverless GPU computing.

OlmOCR's methodology includes a technique called "Document Anchoring," where text and metadata extracted from the source PDF (using tools like PyPDF) are concatenated and inserted into the model's prompt to provide additional context and improve accuracy.[6] While effective, this approach inherently increases the length of the prompt sent to the VLM for each page.

ROlmOCR, in contrast, is explicitly designed as a "faster, lighter" drop-in alternative by **eliminating the dependency on these metadata inputs**.[11] This seemingly small architectural change has a profound and cascading positive impact on its suitability for a serverless environment:

1. **Reduced VRAM Consumption:** In transformer-based models, a significant portion of VRAM during inference is consumed by the KV cache, which stores attention keys and values for each token in the prompt. By significantly reducing the prompt length, ROlmOCR requires a smaller KV cache per request. This lowers the overall VRAM footprint, potentially allowing the model to run on smaller, more cost-effective GPU instances or enabling larger batch sizes on the same hardware.
2. **Faster Inference Time:** The computational cost of inference scales with the length of the input sequence. Shorter prompts mean fewer tokens to process, resulting in faster processing time per document. In a serverless model where billing is metered per second, this directly translates to lower compute costs per inference.
3. **Simplified Pre-processing Pipeline:** The data preparation stage is streamlined. The need to run a separate process to extract text blocks and metadata from the PDF is eliminated, reducing both computational overhead and pipeline complexity.

The evolution from OlmOCR to ROlmOCR demonstrates a critical trend in applied AI: the optimization of models not just for benchmark accuracy, but for production efficiency. ROlmOCR makes a deliberate trade-off, betting that the performance gains and cost reductions from a lighter architecture outweigh any minor accuracy variations from omitting metadata. For a serverless deployment, where cost and latency are paramount, this is the correct strategic choice. Therefore, the remainder of this analysis will focus on the deployment of ROlmOCR.

3.0 The Inference Engine: Maximizing Throughput with vLLM

The choice of inference engine is as critical as the choice of model for achieving the user's "high-throughput" objective. The vLLM library is not merely an implementation detail; it is a core strategic component that unlocks significant performance and efficiency gains, fundamentally shaping the requirements for the underlying serverless platform.

3.1 Core Technical Advantages for OCR Workloads

vLLM is an open-source library engineered from the ground up for fast, high-throughput, and memory-efficient LLM inference and serving.[19] Its primary innovations, PagedAttention and continuous batching, are particularly well-suited to the variable nature of an OCR workload.

- **PagedAttention:** The library's core technical breakthrough is an attention algorithm named PagedAttention, which draws inspiration from virtual memory and paging techniques used in modern operating systems.[19] Traditional inference systems store the attention key-value (KV) cache—a memory-intensive component that grows with sequence length—in contiguous memory blocks. This leads to significant memory waste through internal fragmentation (when reserved blocks are not fully used) and external fragmentation (when free memory is scattered in small, unusable chunks). PagedAttention solves this by partitioning the KV cache into smaller, fixed-size blocks that can be stored non-contiguously in memory.[19] For an OCR workload, where each document page represents a sequence of variable length and complexity, this is transformative. It allows the system to pack requests with different sequence lengths together far more efficiently, achieving near-zero memory waste in the KV cache and dramatically increasing the effective batch size that can fit into a GPU's VRAM.
- **Continuous Batching:** Building on the efficiency of PagedAttention, vLLM implements continuous batching of incoming requests.[20] In older static batching systems, the server must wait to accumulate a full batch of requests before starting computation, leading to idle GPU time and higher latency for individual requests. vLLM, by contrast, processes requests as they arrive. As soon as any sequence in the current batch finishes generation, a new sequence from the waiting queue can be immediately scheduled to take its place. This ensures the GPU is almost constantly active, maximizing utilization. The result is a significant improvement in serving throughput—cited as **2-4 times higher** than state-of-the-art systems like FasterTransformer—while simultaneously reducing average latency.[19] This dual benefit is ideal for both interactive, real-time OCR APIs and large-scale, offline batch processing jobs.

3.2 Deployment Paradigm and Platform Requirements

The design of vLLM and its ecosystem has led to a standardized and highly portable deployment pattern, which in turn defines the essential capabilities required from any serverless platform.

- **OpenAI-Compatible API Server:** A key feature of vLLM is its ability to be launched as a standalone server that exposes an OpenAI-compatible API endpoint.[20] This is achieved with a simple command, such as

vllm serve reducto/RolmOCR.[10] The significance of this cannot be overstated. It provides a universal, well-documented, and industry-standard interface for any client application to interact with the deployed model. This decouples the application logic from the specific serving infrastructure and allows for seamless integration with a vast ecosystem of tools, including popular frameworks like LangChain and LlamaIndex, which are built to communicate with OpenAI-compatible endpoints.[22]

- **Dockerization as the Standard:** The canonical method for deploying a vLLM server is within a Docker container. The vLLM project provides official base images, such as vllm/vllm-openai:latest, which can be used to serve any specified model from a repository like Hugging Face.[10] This container-centric approach encapsulates all dependencies and ensures a consistent runtime environment. Consequently, any serverless platform considered for this task
  must provide first-class, robust support for deploying custom Docker containers from public or private registries.
- **Granular Environment Control:** The deployment process often requires specific configurations passed as environment variables. For instance, the guide for deploying ROlmOCR on Vast.ai explicitly requires setting the environment variable VLLM_USE_V1=1 to ensure compatibility with a specific version of the vLLM engine.[10] This underscores the need for a platform that allows for granular control over the container's runtime environment, including the ability to inject custom environment variables and secrets.

The combination of vLLM's standardized API and its container-based deployment model effectively commoditizes the model-serving layer. The complex challenge of building a high-performance, scalable inference server is largely solved by the vLLM software itself. This shifts the focus of the platform evaluation away from proprietary, high-level MLOps features (which are rendered redundant by this stack) and squarely onto the fundamental capabilities of the underlying infrastructure. The key differentiating factors between platforms become their raw performance on core metrics: how quickly they can start a container, the cost and availability of the required GPUs, and the ease with which a custom vLLM container can be deployed and managed.

Furthermore, the technical advantages of vLLM and ROlmOCR are mutually reinforcing. The architectural decision in ROlmOCR to reduce prompt length by omitting metadata directly decreases the size of the KV cache required for each request. This smaller memory footprint per request allows vLLM's PagedAttention memory manager to pack an even greater number of concurrent requests into the same amount of VRAM. This synergy creates a compounding effect on efficiency: a more memory-efficient model running on a more memory-efficient serving engine results in a dramatic increase in overall throughput per dollar, strengthening the case for deploying this specific combination on a platform that can run it without modification.

4.0 Technical Assessment of Leading Serverless GPU Platforms

This section provides a head-to-head technical comparison of the most viable serverless GPU platforms, evaluated against the specific requirements of the ROlmOCR+vLLM workload as defined in the preceding sections. The analysis focuses on container support, hardware availability, and, most critically, performance characteristics related to latency and cold starts.

4.1 RunPod

- **Platform Architecture:** RunPod is an AI-native cloud platform built with a container-first philosophy. It offers two primary compute products: persistent "Pods," which are akin to GPU-powered virtual machines, and ephemeral "Serverless" endpoints, which are designed for on-demand, scalable inference workloads.[24] The serverless offering aligns perfectly with the target use case, allowing workloads to scale to zero and back automatically. Deployment is managed through a web console, API, or CLI, and is centered around the concept of deploying a Docker image to a configurable endpoint.[24]
- **Custom Container & vLLM Deployment:** Support for this workflow is excellent and a primary feature of the platform. Users can deploy any custom Docker image from a public or private registry like Docker Hub.[25] RunPod provides extensive documentation and dedicated templates specifically for deploying vLLM workers, including a "Quick Deploy" option in the console that simplifies the process of serving a model from Hugging Face.[27] The platform offers full control over the container environment, allowing the injection of public environment variables (such as MODEL_NAME or QUANTIZATION) and secrets, which is necessary for setting configurations like VLLM_USE_V1=1 and providing Hugging Face tokens for gated models.[29]
- **Hardware Availability:** RunPod offers one of the most extensive selections of NVIDIA GPUs on the market. The available hardware ranges from consumer-grade cards like the RTX 3090 (24GB) and RTX 4090 (24GB) to a full suite of data-center-grade accelerators, including the A40 (48GB), A100 (80GB), L40S (48GB), H100 (80GB), and the latest B200 and H200 GPUs.[1] For the ROlmOCR workload, multiple suitable and cost-effective options are readily available, such as the L4 (24GB), RTX 4090 (24GB), and A100 (80GB), providing flexibility to balance performance and cost.[32]
- **Performance & Latency:** This is RunPod's most significant technical differentiator. The platform has invested heavily in minimizing cold-start latency. Official statistics claim that **48% of all serverless cold starts on the platform complete in under 200ms**.[1] This

remarkable performance is achieved through proprietary technologies like **FlashBoot**, which enables lightning-fast scaling.[32] For applications where even minimal latency is unacceptable, RunPod offers "Active" workers. These are always-on, pre-warmed instances that eliminate cold starts entirely for a continuous fee, while still allowing the endpoint to scale up with "Flex" workers to handle traffic spikes.[32] Even standard cold starts for larger containers are cited as being between 6–12 seconds, with some documentation suggesting times as low as 2-5 seconds.[30]

4.2 Modal

- **Platform Architecture:** Modal is a serverless platform designed with a Python-native, code-first developer experience. Instead of managing Dockerfiles and container registries, developers define their entire infrastructure—including dependencies, hardware requirements, and secrets—directly within their Python scripts using the Modal SDK.[1] Modal's backend automatically translates this Python code into an optimized container image and deploys it. This approach is highly opinionated but offers an exceptionally streamlined workflow for teams comfortable within the Python ecosystem.[37]
- **Custom Container & vLLM Deployment:** The platform fully supports the deployment of complex applications like a vLLM server. A developer defines a modal.Image object and uses chained methods like .pip_install("vllm") or .run_commands(...) to construct the environment.[36] It is also possible to start from an existing base Docker image. The Modal documentation and community examples feature numerous guides for deploying vLLM, demonstrating that it is a common and well-supported use case.[39] The platform includes a built-in system for managing secrets, which is ideal for securely providing API keys or Hugging Face tokens to the container environment.[39]
- **Hardware Availability:** Modal provides access to a wide array of modern NVIDIA GPUs, including the T4, L4, A10, L40S, A100 (40GB and 80GB), H100, H200, and B200.[1] This extensive selection ensures that users can choose the optimal GPU for the ROlmOCR workload, from cost-effective L4s to high-performance A100s.
- **Performance & Latency:** Modal's advertised cold-start times are typically between **2-4 seconds**.[1] The platform has heavily optimized its custom container runtime for speed, claiming it is significantly faster than standard Docker.[35] A key innovation recently introduced is
  GPU snapshotting. This feature leverages NVIDIA's checkpoint/restore API to create a snapshot of the GPU's memory state after the model weights have been loaded. When a new container starts, it can restore this snapshot directly into VRAM, bypassing the time-consuming process of loading the model from disk. This can reduce cold boot times by up to 12x and is a powerful tool for mitigating the primary source of latency for large

models like ROlmOCR.[43]

4.3 Google Cloud Run

- **Platform Architecture:** As a flagship serverless offering from a major hyperscaler, Google Cloud Run is a fully managed, general-purpose platform designed to run any stateless container that listens for HTTP requests.[44] It is deeply integrated with the broader Google Cloud ecosystem, including services like Artifact Registry, Cloud Build, and Secret Manager.[47]
- **Custom Container & vLLM Deployment:** Support for custom containers is a core feature. Users can build a container image using a Dockerfile and Cloud Build, or push a pre-built image to Google's Artifact Registry, and then deploy it to a Cloud Run service via the gcloud command-line tool or the web console.[45] The workflow is robust and well-documented, with official Google codelabs available that specifically guide users through deploying vLLM on Cloud Run, confirming that this is a fully supported pathway.[48]
- **Hardware Availability:** This is a significant limitation compared to the specialized AI platforms. The serverless GPU offering on Cloud Run is currently limited to the **NVIDIA L4 GPU**, which provides 24GB of VRAM.[1] While the L4 GPU meets the minimum VRAM requirements for ROlmOCR and is a capable inference accelerator, the lack of access to higher-end options like the A100 or H100 could be a constraint for future, larger models or for workloads that demand maximum throughput.
- **Performance & Latency:** Google advertises a rapid startup time, stating that an instance with a GPU and drivers installed can be ready in **under 5 seconds**.[52] However, this figure is for the instance boot time alone and does not include the critical model-loading phase. A more realistic metric provided by Google is the "Time-to-First-Token," which they report as approximately 19 seconds for a smaller 4B parameter model.[52] Independent user testing has shown that full cold starts for larger models can be significantly longer, in the range of **105-120 seconds**, with "warm" starts (when an instance is reused within a short time frame) taking around 6-7 seconds.[53] This high variability and potentially long latency for a true cold start is a major drawback for any latency-sensitive application. Cloud Run offers mitigation strategies like "startup CPU boost" and configuring "minimum instances" to keep one or more instances always warm, but these add cost and complexity to the deployment.[54]

4.4 Azure Container Apps

- **Platform Architecture:** Azure Container Apps is Microsoft's serverless container platform, analogous to Google Cloud Run. It is built on an underlying Kubernetes foundation and integrates with open-source technologies like KEDA (Kubernetes Event-driven Autoscaling) and Dapr to provide a rich feature set for building microservices.[55]
- **Custom Container & vLLM Deployment:** The platform offers full support for deploying custom container images from any compatible registry. Deployments are configured via the Azure CLI or the Azure portal.[57] To access GPUs, users must configure the environment to use a GPU-enabled "Workload Profile".[59] The process is well-documented and comparable in complexity to deploying on Google Cloud Run.
- **Hardware Availability:** Azure provides a better selection of GPUs in its serverless "Consumption" plan than Google Cloud Run. It offers the **NVIDIA T4, L4, and A100** GPUs.[1] The availability of the powerful NVIDIA A100 is a significant advantage, providing a clear upgrade path for more demanding workloads and future-proofing the deployment.
- **Performance & Latency:** Microsoft announced serverless GPU support in 2024, with an advertised cold start time of **5 seconds**.[1] As with Google Cloud Run, this number likely represents the instance provisioning time and does not account for the time required to download and load the ROlmOCR model into VRAM. As a newer entrant to the serverless GPU market, there is less publicly available, real-world performance data compared to more established competitors. The performance characteristics can be expected to be broadly similar to those of Cloud Run, with true cold starts for large models taking substantially longer than the advertised 5-second figure.

4.5 Other Contenders (Replicate, DigitalOcean)

- **Replicate:** This platform is primarily designed as a model-sharing and API-hosting service that prioritizes ease of use for a vast library of public, pre-trained models.[1] Its deployment workflow is centered on an open-source tool called **Cog**, which packages a model and its dependencies into a standard container format.[62] While it is possible to deploy a custom vLLM server using Cog, this is a less direct workflow compared to platforms like RunPod or Modal. A significant drawback is that cold starts for custom models can be quite slow, with documentation noting they can take **up to 1 minute**.[1] This makes it less suitable for latency-sensitive applications.
- **DigitalOcean:** DigitalOcean's serverless GPU offerings are part of its **Gradient™ AI Agentic Cloud**.[1] The platform's documentation emphasizes its higher-level PaaS (Platform as a Service) and FaaS (Function as a Service) products, such as "App Platform" and "Functions".[64] While App Platform can deploy services from container images, the overall ecosystem appears more geared towards either fully managed AI

workflows or traditional IaaS via "GPU Droplets" (virtual machines with hourly billing).[67] The path to deploying a high-performance, custom vLLM server in a truly serverless, scale-to-zero model is less clear and less documented than on the specialized AI platforms.

The technical evaluation reveals a distinct trade-off between the developer experience (DX) and the level of infrastructure control. Modal represents the high-DX end of the spectrum, abstracting away containerization into a Python-native SDK.[36] RunPod, Google Cloud Run, and Azure Container Apps operate at the container level, offering greater control at the cost of requiring more direct DevOps and infrastructure management.[26] Replicate abstracts even further to the model level, offering the highest DX for supported models but the least flexibility for custom deployments.[63] The optimal choice depends on a team's existing skill set and priorities.

A more critical finding, however, is the inconsistency and potential for misinterpretation surrounding the term "cold start." Marketing claims from various providers often refer to different stages of the startup process. The actual user-experienced latency, or "Time to First Token," is a function of the entire startup chain: instance provisioning, container boot, inference server initialization, and, most importantly, loading the multi-gigabyte model weights into VRAM. The evidence shows that instance boot time (the 5-second figure from GCP/Azure) is a small and often misleading component of the total cold-start penalty for large models, which can exceed 100 seconds.[52] Therefore, platforms that have developed specific technologies to attack the model-loading bottleneck—such as Modal's GPU snapshotting and RunPod's FlashBoot and active worker pools—hold a fundamental performance advantage that is far more significant than marginal differences in container boot times.

4.6 Technical Capabilities Matrix

The following table synthesizes the technical assessment, providing a comparative overview of the leading platforms against the key requirements for deploying the ROlmOCR+vLLM workload.

Platform	Primary Deployment Model	Custom Container Support	vLLM Deployment Ease	Available NVIDIA GPUs (Relevant Models)	Advertised Cold Start	Key Performance Technology

RunPod	Container-first	Excellent	Excellent (Templates available)	L4, RTX 4090, A40, L40S, A100, H100, B200	<200ms (48% of starts)	FlashBoot, Active (pre-warmed) Workers
Modal	Code-first (Python)	Excellent (via SDK)	Excellent (Examples available)	T4, L4, A10, L40S, A100, H100, B200	2-4 seconds	GPU Snapshotting, Optimized Container Runtime
Google Cloud Run	Container-first	Excellent	Good (Codelabs available)	L4	<5 seconds (instance only)	Startup CPU Boost, Minimum Instances
Azure Container Apps	Container-first	Excellent	Good	L4, T4, A100	~5 seconds (instance only)	Workload Profiles
Replicate	Model-first (via Cog)	Good (via Cog)	Fair (Less direct)	T4, L40S, A100, H100	Up to 60 seconds (custom models)	Cog Containerization

5.0 Commercial Assessment and Total Cost of Ownership (TCO) Analysis

While technical feasibility is a prerequisite, the commercial viability and predictability of a

solution are paramount for a production deployment. This section analyzes the economic aspects of each platform, moving beyond headline GPU prices to construct a more holistic Total Cost of Ownership (TCO) model that includes ancillary but significant cost factors.

5.1 Direct Compute Cost Comparison (GPU Pricing)

The most direct cost is the price of GPU compute time. A crucial feature for serverless workloads is fine-grained billing, as it ensures costs are tightly coupled with actual usage. All the primary contenders—RunPod, Modal, Google Cloud Run, Azure Container Apps, and Replicate—offer **per-second billing**, which is the industry standard and ideal for ephemeral inference tasks.[1]

To facilitate a direct comparison, the following are the approximate on-demand, pay-as-you-go prices for relevant GPU types, normalized to a cost per hour:

- **RunPod:** Highly competitive, especially in its "Community Cloud." Serverless pricing for a 24GB VRAM GPU (L4, A5000, 3090) is $0.00019/s (~$0.68/hr), and for an 80GB A100 it is $0.00076/s (~$2.74/hr).[32] Standard Pod (VM) pricing can be even lower, with an A100 PCIe at ~$1.19/hr.[31]
- **Modal:** Pricing is competitive and straightforward. An L4 GPU costs $0.000222/s (~$0.80/hr), and an 80GB A100 costs $0.000694/s (~$2.50/hr).[41]
- **Google Cloud Run:** The NVIDIA L4 GPU is priced at $0.0002909/s (~$1.05/hr) when configured with zonal redundancy.[73] It is critical to note that this price is for the GPU resource
  only; CPU and memory are billed as separate line items.
- **Azure Container Apps:** Offers compelling rates on its serverless plan. An NVIDIA T4 is priced at $0.00009/s (~$0.32/hr), and an NVIDIA A100 is $0.000651/s (~$2.34/hr).[70] Similar to Google, these prices are additive to the costs of vCPU and memory resources consumed by the container.
- **Replicate:** Tends to be positioned at a premium price point for high-end hardware. An NVIDIA T4 costs $0.000225/s (~$0.81/hr), while an 80GB A100 is significantly more expensive at $0.001400/s (~$5.04/hr).[71]
- **Vast.ai:** As a peer-to-peer marketplace, prices are dynamic but consistently among the lowest available. An A100 PCIe can be found for as low as $0.67/hr.[74] However, this comes with potential trade-offs in reliability and availability compared to enterprise-grade cloud providers.

5.2 Ancillary and "Hidden" Cost Factors

A TCO analysis based solely on GPU prices is incomplete and potentially misleading. Several other factors can contribute significantly to the final monthly bill, and the way platforms handle these costs reveals fundamental differences in their business models.

- **CPU and Memory Costs:** This is a critical distinction between the specialized AI platforms and the hyperscalers.
  - **Hyperscalers (GCP, Azure):** Both Google Cloud Run and Azure Container Apps follow an "à la carte" pricing model. Users are billed separately for GPU time, vCPU-seconds, and GiB-seconds of memory.[70] While this offers granular control, it makes cost forecasting more complex and can lead to unexpectedly high bills if CPU or memory usage is not carefully managed.
  - **Specialized Platforms (RunPod, Modal):** These platforms typically offer a bundled pricing model. The per-second rate for a specific GPU instance type (e.g., "A100") includes the associated CPU cores and system RAM.[32] This all-inclusive approach provides much greater cost predictability.
- **Persistent Storage:** To minimize cold-start latency, it is essential to store the multi-gigabyte model weights in a fast, persistent storage volume rather than downloading them from Hugging Face on every startup.
  - **RunPod:** Offers "Network Volumes" with a simple pricing structure: $0.07/GB/month for storage under 1TB, and $0.05/GB/month for storage over 1TB.[31]
  - **Modal:** Provides a built-in distributed storage layer called modal.Volume.[36] The pricing for this is not explicitly broken out and appears to be bundled into the overall platform cost, simplifying billing.
  - **GCP/Azure:** Would require provisioning and attaching a native storage solution, such as Google Cloud Storage or Azure Blob Storage. This introduces additional costs for storage capacity (e.g., GCS standard storage is ~$0.020/GB/month) as well as charges for data access operations (e.g., Class A/B operations).[78]
- **Data Transfer (Egress/Ingress):** For a data-intensive OCR pipeline that involves uploading potentially large documents and downloading structured text output, data transfer costs can be a major, often overlooked, expense.
  - **RunPod:** This is a standout commercial advantage. RunPod explicitly states a policy of **zero fees for data ingress or egress**.[80] This completely eliminates a significant and variable component of the TCO.
  - **Modal:** The platform's pricing documentation does not explicitly detail data transfer charges, which suggests they are either bundled into the compute cost or are considered negligible.[41] This contrasts sharply with the explicit charges levied by hyperscalers.
  - **GCP/Azure:** These platforms follow the standard cloud model where data ingress is generally free, but data egress (data transfer out to the internet) is metered and

charged per gigabyte. For example, Google Cloud's Premium Tier egress starts at **$0.12 per GiB** after a small monthly free tier.[73] For a high-volume OCR service processing thousands of multi-page documents, these charges could accumulate to become a substantial portion of the monthly bill.

The differing approaches to these ancillary costs highlight a fundamental divergence in business models. The specialized AI clouds like RunPod and Modal offer a more predictable, all-inclusive TCO profile, which is highly attractive for startups and businesses seeking to control costs. In contrast, the hyperscalers' unbundled, "à la carte" model provides more granular control but carries the risk of unforeseen expenses, particularly from data transfer, which can make budgeting more challenging. A simple comparison of hourly GPU rates is therefore dangerously incomplete; the "cheaper" GPU on a hyperscaler could easily result in a higher final bill once all associated costs are factored in.

5.3 Pricing Model Nuances

Beyond the primary cost drivers, several platform-specific features and pricing models can influence the overall economic equation.

- **Free Tiers and Promotional Credits:** Many platforms offer incentives to attract new users. Modal provides a $30/month free compute credit on its starter plan, Azure Container Apps includes a monthly free grant of vCPU-seconds and requests, and Google Cloud offers a $300 credit for new customers.[35] While valuable for initial development, testing, and proof-of-concept work, these temporary credits should not be factored into a long-term production TCO model.
- **Hybrid Serverless Models (Active vs. Idle):** The challenge of cold-start latency has led to the emergence of hybrid pricing models that blend serverless scaling with the reliability of a reserved instance.
    - **RunPod** formalizes this with its "Active" vs. "Flex" worker types. "Flex" workers are the standard scale-to-zero model. "Active" workers are always-on, pre-warmed instances that eliminate cold starts. They are billed continuously but at a rate that can be up to 30% lower than the "Flex" rate, making them a cost-effective way to guarantee performance for a baseline level of traffic.[32]
    - **GCP and Azure** achieve a similar outcome by allowing users to configure a minimum instances setting of 1 or more.[45] These idle-but-warm instances are typically billed at a reduced rate for CPU and memory compared to active instances.[73]

This hybrid approach signifies a maturation of the serverless market. It acknowledges that for many real-world, latency-sensitive AI applications, a "pure" scale-to-zero model is not performant enough. The optimal commercial strategy often involves paying a small, fixed cost

for a single warm instance to handle baseline traffic and absorb initial request spikes, while leveraging the elasticity of serverless scaling to handle unpredictable peak loads. The platform that prices this hybrid model most attractively and transparently holds a significant commercial advantage.

5.4 Comprehensive Commercial Cost Comparison

The following table summarizes the key commercial aspects of each platform, providing a holistic view of the TCO beyond raw GPU compute costs.

Platform	Billing Unit	GPU Price (L4/hr, Norm.)	GPU Price (A100/hr, Norm.)	CPU/RAM Pricing	Persistent Storage (/GB/mo)	Data Egress (/GB)	Key Commercial Advantage
RunPod	Per-second	~$0.68	~$2.74	Bundled	$0.07 (<1TB)	**$0.00**	Zero egress fees; highly predictable TCO.
Modal	Per-second	~$0.80	~$2.50	Bundled	Bundled	Not specified (likely bundled)	Simplicity; bundled costs and dev credits.
Google Cloud Run	Per-second	~$1.05	N/A	Separate	~$0.02 + ops	~$0.12	Deep ecosystem integration; CUDs.

Azure Container Apps	Per-second	N/A	~$2.34	Separate	~$0.023 + ops	~$0.085	Competitive GPU rates; free tier.
Replicate	Per-second	~$0.81	~$5.04	Bundled	Not specified	Not specified	Ease of use for public models.

6.0 Integrated Analysis and Strategic Recommendations

This final section synthesizes the preceding technical and commercial analyses to provide clear, actionable recommendations. The optimal choice of platform is not absolute but depends on the specific priorities of the deploying organization, whether they be raw performance, cost-effectiveness, or developer velocity.

6.1 Scenario-Based Recommendations

To provide the most relevant guidance, the recommendations are structured around three common deployment scenarios, each with a different primary optimization goal.

Scenario A: Optimal for Real-Time, Latency-Sensitive Applications

- **Use Case:** An interactive web application where users upload documents and expect immediate OCR results.
- **Primary Goal:** Minimize "Time to First Token" and ensure a consistently responsive user experience.

- **Recommendation: RunPod** configured with a minimum of one "Active" worker.
- **Justification:** For this scenario, eliminating cold-start latency is the highest priority. RunPod's "Active" worker model is explicitly designed for this purpose, keeping a pre-warmed instance ready to process requests instantly, thereby removing the cold-start penalty entirely for the first user.[32] For handling concurrent requests that require scaling up, its FlashBoot technology provides the fastest scale-up latency in the market, with a significant portion of new instances starting in under 200ms.[33] This combination ensures both initial responsiveness and scalable performance. From a commercial standpoint, the predictable, all-inclusive pricing model and the critical advantage of zero egress fees de-risk the deployment and prevent unpredictable costs associated with high traffic. The technical overhead of preparing a custom Docker container is a minor and justifiable trade-off for achieving this level of production-grade performance.

Scenario B: Optimal for Cost-Effective, High-Throughput Batch Processing

- **Use Case:** A large-scale, asynchronous project to digitize an archive of millions of PDF documents stored in a cloud bucket.
- **Primary Goal:** Minimize the total cost per processed page.
- **Recommendation: RunPod** with "Flex" workers or **Vast.ai**.
- **Justification:** In a batch processing context, the cost per inference is the dominant metric, and a startup latency of several seconds or even a minute per batch job is generally acceptable.
    - **RunPod's** "Flex" worker model provides a pure pay-as-you-go service with extremely competitive GPU rates and, crucially, no data transfer costs.[32] For a workload that involves processing massive amounts of data from a source like AWS S3, the elimination of egress fees can result in substantial savings, making RunPod a top contender for TCO. The platform's native support for the OlmOCR toolkit's multi-node S3 processing pattern makes it a natural fit.[5]
    - **Vast.ai**, with its peer-to-peer marketplace model, often offers the absolute lowest raw GPU prices available.[74] For a purely cost-driven project, this can be the most economical option. However, this comes with the caveat of potentially variable reliability and instance availability compared to an enterprise-grade provider. This option is best suited for workloads that are highly fault-tolerant and where cost is the single most important factor.

Scenario C: Optimal for Developer Velocity and Rapid Prototyping

- **Use Case:** A startup or an R&D team aiming to quickly build and iterate on an AI-powered feature without a dedicated DevOps team.
- **Primary Goal:** Minimize time-to-market and reduce infrastructure complexity for developers.
- **Recommendation: Modal**.
- **Justification:** For teams where developer time is the most valuable resource, Modal's SDK-first, Python-native approach is unparalleled.[35] It abstracts away the complexities of Docker, container registries, and infrastructure configuration, allowing a developer to move from a local Python script to a scalable, GPU-powered API endpoint with minimal friction.[38] Features like integrated persistent storage (modal.Volume) and built-in secret management further streamline the development process.[38] While its raw compute cost may not be the absolute lowest, the significant reduction in development time and cognitive overhead can provide a decisive competitive advantage. The recent introduction of GPU snapshotting technology makes its performance highly competitive, mitigating much of the traditional cold-start penalty.[43]

6.2 Final Recommendation and Justification

Across the evaluated scenarios, **RunPod** consistently emerges as the most balanced, powerful, and commercially sound platform for this specific workload. It achieves the highest scores across the most critical evaluation vectors:

1. **Performance (Latency):** It offers the most effective and flexible solutions for cold-start mitigation in the market, from the ultra-fast FlashBoot for on-demand scaling to Active workers for complete latency elimination.
2. **Cost (TCO):** Its combination of competitive per-second GPU pricing, bundled CPU/memory, and a zero-egress-fee policy provides the most predictable and likely the lowest total cost of ownership, especially for data-intensive applications.
3. **Flexibility:** It provides native, low-level support for the required Docker/vLLM stack without forcing developers into a specific, proprietary programming paradigm, offering maximum control and portability.

While Modal excels in developer experience and the hyperscalers offer deep ecosystem integration, RunPod's focused and superior execution on solving the core problems of serverless GPU computing—performance and cost—makes it the most robust choice for a production-grade deployment of ROlmOCR with vLLM.

6.3 Implementation Blueprint on Recommended Platform (RunPod)

The following steps outline a high-level blueprint for deploying the RolmOCR model on RunPod Serverless:

1. **Containerize the Application:** Create a Dockerfile to build the inference server. The recommended approach is to use an official vLLM worker image, such as runpod/worker-v1-vllm, as a base.[29] This image already contains the necessary logic to start a vLLM server. Configure the container to use the target model by setting the MODEL_NAME environment variable to reducto/RolmOCR. Crucially, also set the VLLM_USE_V1=1 environment variable as required by the model.[10] If the model were private, a Hugging Face token would be passed as a Docker secret during the build process.[29] After building, push the final image to a container registry like Docker Hub.
2. **Set Up Persistent Storage for Model Weights:** To achieve the fastest possible initialization times, pre-download the model weights instead of fetching them on every cold start. Create a RunPod Network Volume in the RunPod console. Launch a temporary RunPod Pod (a standard VM), attach this volume, and use it to download and cache the reducto/RolmOCR model files from Hugging Face. Once cached, terminate the temporary Pod. The weights will remain on the persistent Network Volume.[25]
3. **Create a Serverless Endpoint:** In the RunPod console, navigate to the "Serverless" section and create a new endpoint. Select the "Import from Docker Registry" option and provide the path to the custom container image created in step 1.[26]
4. **Configure the Endpoint:** In the endpoint configuration settings, select an appropriate GPU type based on performance and cost requirements (e.g., an NVIDIA L4 or RTX 4090 for a balance, or an A100 for maximum throughput). Set the desired scaling parameters, such as the minimum and maximum number of workers. For a latency-sensitive application, set the "Min Workers" to 1 to enable the "Active" worker mode. In the "Environment Variables" section, ensure all necessary variables are set. While the model name can be baked into the image, it can also be set here for flexibility.
5. **Deploy and Test:** Deploy the endpoint. RunPod will provision the resources and provide a unique API endpoint URL. Using this URL and a RunPod API key, send test requests to the endpoint. Since the vLLM worker exposes an OpenAI-compatible interface, requests can be formatted identically to calls to the OpenAI Chat Completions API, simplifying client-side integration.[29] Monitor the endpoint's logs and performance metrics (requests per second, latency, cost) directly in the RunPod dashboard to validate the deployment and fine-tune scaling parameters.

Final Recommendation Scorecard

The table below provides a final, high-level summary that visually justifies the recommendation by scoring each leading platform against the most critical evaluation criteria identified in this report.

Criteria	RunPod	Modal	Google Cloud Run
GPU Cost-Effectiveness	Excellent	Good	Fair
TCO Predictability (Incl. Data Transfer)	Excellent	Good	Poor
Cold-Start Performance	Excellent	Good	Fair
Custom vLLM Deployment	Excellent	Excellent	Good
Developer Experience	Good	Excellent	Good
Hardware Selection	Excellent	Excellent	Poor

Works cited

1. 7 Serverless GPU Platforms for Scalable Inference Workloads | DigitalOcean, accessed on October 5, 2025, https://www.digitalocean.com/resources/articles/serverless-gpu-platforms
2. allenai/olmOCR-7B-0225-preview - Hugging Face, accessed on October 5, 2025, https://huggingface.co/allenai/olmOCR-7B-0225-preview
3. allenai/olmOCR-7B-0725 - Hugging Face, accessed on October 5, 2025, https://huggingface.co/allenai/olmOCR-7B-0725
4. Show HN: We Built a Serverless GPU Platform with Fast Cold Starts | Hacker News, accessed on October 5, 2025, https://news.ycombinator.com/item?id=44745428

5. allenai/olmocr: Toolkit for linearizing PDFs for LLM datasets/training - GitHub, accessed on October 5, 2025, https://github.com/allenai/olmocr
6. Efficient PDF Text Extraction with Vision Language Models - olmOCR - Ai2, accessed on October 5, 2025, https://olmocr.allenai.org/blog
7. olmOCR Open Source OCR System for AI Training Using PDFs & Documents, accessed on October 5, 2025, https://www.geeky-gadgets.com/olmocr-ocr-model-for-ai-training/
8. Unlocking Trillions of Tokens in PDFs with Vision Language Models - olmOCR, accessed on October 5, 2025, https://olmocr.allenai.org/papers/olmocr.pdf
9. Reducto AI Released RolmOCR: A SoTA OCR Model Built on Qwen 2.5 VL, Fully Open-Source and Apache 2.0 Licensed for Advanced Document Understanding : r/machinelearningnews - Reddit, accessed on October 5, 2025, https://www.reddit.com/r/machinelearningnews/comments/1jsmpds/reducto_ai_released_rolmocr_a_sota_ocr_model/
10. Running RolmOCR on Vast, accessed on October 5, 2025, https://vast.ai/article/running-rolmocr-on-vast
11. Introducing RolmOCR: A Faster, Lighter Open Source Document Model Built on olmOCR - Reducto, accessed on October 5, 2025, https://reducto.ai/blog/introducing-rolmocr-open-source-ocr-model
12. RolmOCR - PromptLayer, accessed on October 5, 2025, https://www.promptlayer.com/models/rolmocr
13. reducto/RolmOCR - Hugging Face, accessed on October 5, 2025, https://huggingface.co/reducto/RolmOCR
14. Install olmOCR: The Best Open-Source OCR Toolkit for Document Parsing, accessed on October 5, 2025, https://dev.to/nodeshiftcloud/install-olmocr-the-best-open-source-ocr-toolkit-for-document-parsing-npn
15. Installation - vLLM, accessed on October 5, 2025, https://docs.vllm.ai/en/v0.4.2/getting_started/installation.html
16. GPU - vLLM, accessed on October 5, 2025, https://docs.vllm.ai/en/stable/getting_started/installation/gpu.html
17. Installation - vLLM, accessed on October 5, 2025, https://docs.vllm.ai/en/latest/getting_started/installation/index.html
18. olmOCR and RolmOCR: The Latest in Open-Source OCR | DigitalOcean, accessed on October 5, 2025, https://www.digitalocean.com/community/tutorials/olmocr-rolmocr-open-source-models
19. What is vLLM? - Hopsworks, accessed on October 5, 2025, https://www.hopsworks.ai/dictionary/vllm
20. Welcome to vLLM! — vLLM - Read the Docs, accessed on October 5, 2025, https://nm-vllm.readthedocs.io/
21. vLLM - Qwen, accessed on October 5, 2025, https://qwen.readthedocs.io/en/latest/deployment/vllm.html
22. vLLM - LangChain, accessed on October 5, 2025, https://python.langchain.com/docs/integrations/llms/vllm/

23. vLLM | LlamaIndex Python Documentation, accessed on October 5, 2025, https://developers.llamaindex.ai/python/examples/llm/vllm/
24. Overview - Runpod Documentation, accessed on October 5, 2025, https://docs.runpod.io/serverless/overview
25. Overview - Runpod Documentation, accessed on October 5, 2025, https://docs.runpod.io/pods/overview
26. Build your first worker - Runpod Documentation, accessed on October 5, 2025, https://docs.runpod.io/serverless/workers/custom-worker
27. Deploy a vLLM worker - Runpod Documentation, accessed on October 5, 2025, https://docs.runpod.io/serverless/vllm/get-started
28. VLLM worker overview - Runpod Documentation, accessed on October 5, 2025, https://docs.runpod.io/serverless/vllm/overview
29. The RunPod worker template for serving our large language model endpoints. Powered by vLLM. - GitHub, accessed on October 5, 2025, https://github.com/runpod-workers/worker-vllm
30. Top Serverless GPU Clouds for 2025: Comparing Runpod, Modal, and More, accessed on October 5, 2025, https://www.runpod.io/articles/guides/top-serverless-gpu-clouds
31. Pricing | Runpod GPU cloud computing rates, accessed on October 5, 2025, https://www.runpod.io/pricing
32. Serverless GPU Endpoints - Runpod, accessed on October 5, 2025, https://www.runpod.io/product/serverless
33. www.runpod.io, accessed on October 5, 2025, https://www.runpod.io/articles/guides/top-serverless-gpu-clouds#:~:text=Impressive%20Cold%20Start%20Performance%3A%20While,through%20high%2Dend%2080GB%20accelerators.
34. Serverless GPUs for API Hosting: How They Power AI APIs–A Runpod Guide, accessed on October 5, 2025, https://www.runpod.io/articles/guides/serverless-for-api-hosting
35. Modal: High-performance AI infrastructure, accessed on October 5, 2025, https://modal.com/
36. Images | Modal Docs, accessed on October 5, 2025, https://modal.com/docs/guide/images
37. Serverless for AI Devs: Modal's Python and Rust Based Platform - The New Stack, accessed on October 5, 2025, https://thenewstack.io/serverless-for-ai-devs-modals-python-and-rust-based-platform/
38. Deploy with Modal | dlt Docs - dltHub, accessed on October 5, 2025, https://dlthub.com/docs/walkthroughs/deploy-a-pipeline/deploy-with-modal
39. Deploying the Magistral vLLM Server on Modal - KDnuggets, accessed on October 5, 2025, https://www.kdnuggets.com/deploying-the-magistral-vllm-server-on-modal
40. Run OpenAI's gpt-oss model with vLLM | Modal Docs, accessed on October 5, 2025, https://modal.com/docs/examples/gpt_oss_inference
41. Plan Pricing - Modal, accessed on October 5, 2025, https://modal.com/pricing

42. Cold start performance | Modal Docs, accessed on October 5, 2025, https://modal.com/docs/guide/cold-start
43. [D] Implementing GPU snapshotting to cut cold starts for large models by 12x - Reddit, accessed on October 5, 2025, https://www.reddit.com/r/MachineLearning/comments/1mf8d4g/d_implementing_gpu_snapshotting_to_cut_cold/
44. GPU support for services | Cloud Run, accessed on October 5, 2025, https://cloud.google.com/run/docs/configuring/services/gpu
45. Serverless GPU inference on Google Cloud Run - Modular docs, accessed on October 5, 2025, https://docs.modular.com/max/tutorials/deploy-serverless-cloud-run/
46. Cloud Run documentation - Google Cloud, accessed on October 5, 2025, https://cloud.google.com/run/docs
47. Serve dynamic content and host microservices with Cloud Run - Firebase - Google, accessed on October 5, 2025, https://firebase.google.com/docs/hosting/cloud-run
48. Serving Gemma 3 with vLLM on Cloud Run - Google Codelabs, accessed on October 5, 2025, https://codelabs.developers.google.com/devsite/codelabs/serve-gemma3-with-vllm-on-cloud-run
49. Build sources to containers | Cloud Run, accessed on October 5, 2025, https://cloud.google.com/run/docs/building/containers
50. Setting Up vLLM Locally and on Google Cloud for CPU - DataCamp, accessed on October 5, 2025, https://www.datacamp.com/tutorial/vllm
51. Google Cloud Run Adds Support for NVIDIA L4 GPUs, NVIDIA NIM, and Serverless AI Inference Deployments at Scale, accessed on October 5, 2025, https://developer.nvidia.com/blog/google-cloud-run-adds-support-for-nvidia-l4-gpus-nvidia-nim-and-serverless-ai-inference-deployments-at-scale/
52. Cloud Run GPUs are now generally available | Google Cloud Blog, accessed on October 5, 2025, https://cloud.google.com/blog/products/serverless/cloud-run-gpus-are-now-generally-available
53. Serverless GPU Computing: A Technical Deep Dive into CloudRun - DEV Community, accessed on October 5, 2025, https://dev.to/shannonlal/serverless-gpu-computing-a-technical-deep-dive-into-cloudrun-3do2
54. General development tips | Cloud Run - Google Cloud, accessed on October 5, 2025, https://cloud.google.com/run/docs/tips/general
55. Azure Functions on Azure Container Apps overview - Microsoft Learn, accessed on October 5, 2025, https://learn.microsoft.com/en-us/azure/container-apps/functions-overview
56. Azure Container Apps documentation - Microsoft Learn, accessed on October 5, 2025, https://learn.microsoft.com/en-us/azure/container-apps/
57. Azure Container Apps: Your Complete 2025 Guide to Serverless Container Deployment, accessed on October 5, 2025,

https://kunaldaskd.medium.com/azure-container-apps-your-complete-2025-guide-to-serverless-container-deployment-de6ef2ef1f1a
58. Tutorial: Generate images using serverless GPUs in Azure Container Apps | Microsoft Learn, accessed on October 5, 2025, https://learn.microsoft.com/en-us/azure/container-apps/gpu-image-generation
59. Workload profiles in Azure Container Apps | Microsoft Learn, accessed on October 5, 2025, https://learn.microsoft.com/en-us/azure/container-apps/workload-profiles-overview
60. Using serverless GPUs in Azure Container Apps, accessed on October 5, 2025, https://docs.azure.cn/en-us/container-apps/gpu-serverless-overview
61. Documentation - Replicate, accessed on October 5, 2025, https://replicate.com/docs
62. Replicate - Run AI with an API, accessed on October 5, 2025, https://replicate.com/
63. Deploy a custom model - Replicate, accessed on October 5, 2025, https://replicate.com/docs/get-started/deploy-a-custom-model
64. DigitalOcean Functions | Serverless Computing Starting at $0/mo, accessed on October 5, 2025, https://www.digitalocean.com/products/functions
65. Compute | DigitalOcean Documentation, accessed on October 5, 2025, https://docs.digitalocean.com/products/compute/
66. Product Home | DigitalOcean Documentation, accessed on October 5, 2025, https://docs.digitalocean.com/products/
67. GPU Droplets Pricing - DigitalOcean, accessed on October 5, 2025, https://www.digitalocean.com/pricing/gpu-droplets
68. Budget-Friendly Cloud Server Pricing - DigitalOcean, accessed on October 5, 2025, https://www.digitalocean.com/pricing
69. How to Deploy from Container Images | DigitalOcean Documentation, accessed on October 5, 2025, https://docs.digitalocean.com/products/app-platform/how-to/deploy-from-container-images/
70. Azure Container Apps - Pricing, accessed on October 5, 2025, https://azure.microsoft.com/en-us/pricing/details/container-apps/
71. Pricing - Replicate, accessed on October 5, 2025, https://replicate.com/pricing
72. Best 10 Serverless GPU Clouds & 14 Cost-Effective GPUs - Research AIMultiple, accessed on October 5, 2025, https://research.aimultiple.com/serverless-gpu/
73. Cloud Run pricing | Google Cloud, accessed on October 5, 2025, https://cloud.google.com/run/pricing
74. Pricing - Vast AI, accessed on October 5, 2025, https://vast.ai/pricing
75. Rent A100 PCIE GPUs on Vast.ai for $0.67/hr | Vast.ai, accessed on October 5, 2025, https://vast.ai/pricing/gpu/A100-PCIE
76. docs.runpod.io, accessed on October 5, 2025, https://docs.runpod.io/references/faq#:~:text=For%20storage%20requirements%20below%201TB,of%20%240.05%2FGB%2FMonth.
77. Runpod Pricing (September 2025) vs Thunder Compute, accessed on October 5,

2025, https://www.thundercompute.com/blog/runpod-pricing-vs-thunder-compute
78. Google Cloud Cost Breakdown - Umbrella, accessed on October 5, 2025, https://umbrellacost.com/learning-center/google-cloud-cost/
79. Azure Data Lake Storage Pricing, accessed on October 5, 2025, https://azure.microsoft.com/en-us/pricing/details/storage/data-lake/
80. Runpod | The cloud built for AI, accessed on October 5, 2025, https://www.runpod.io/
81. RunPod Software Pricing & Plans 2025: See Your Cost - Vendr, accessed on October 5, 2025, https://www.vendr.com/marketplace/runpod
82. Network pricing - Google Cloud, accessed on October 5, 2025, https://cloud.google.com/vpc/network-pricing
83. Azure Container Apps, accessed on October 5, 2025, https://azure.microsoft.com/en-us/products/container-apps
84. Rent GPUs | Vast.ai, accessed on October 5, 2025, https://vast.ai/